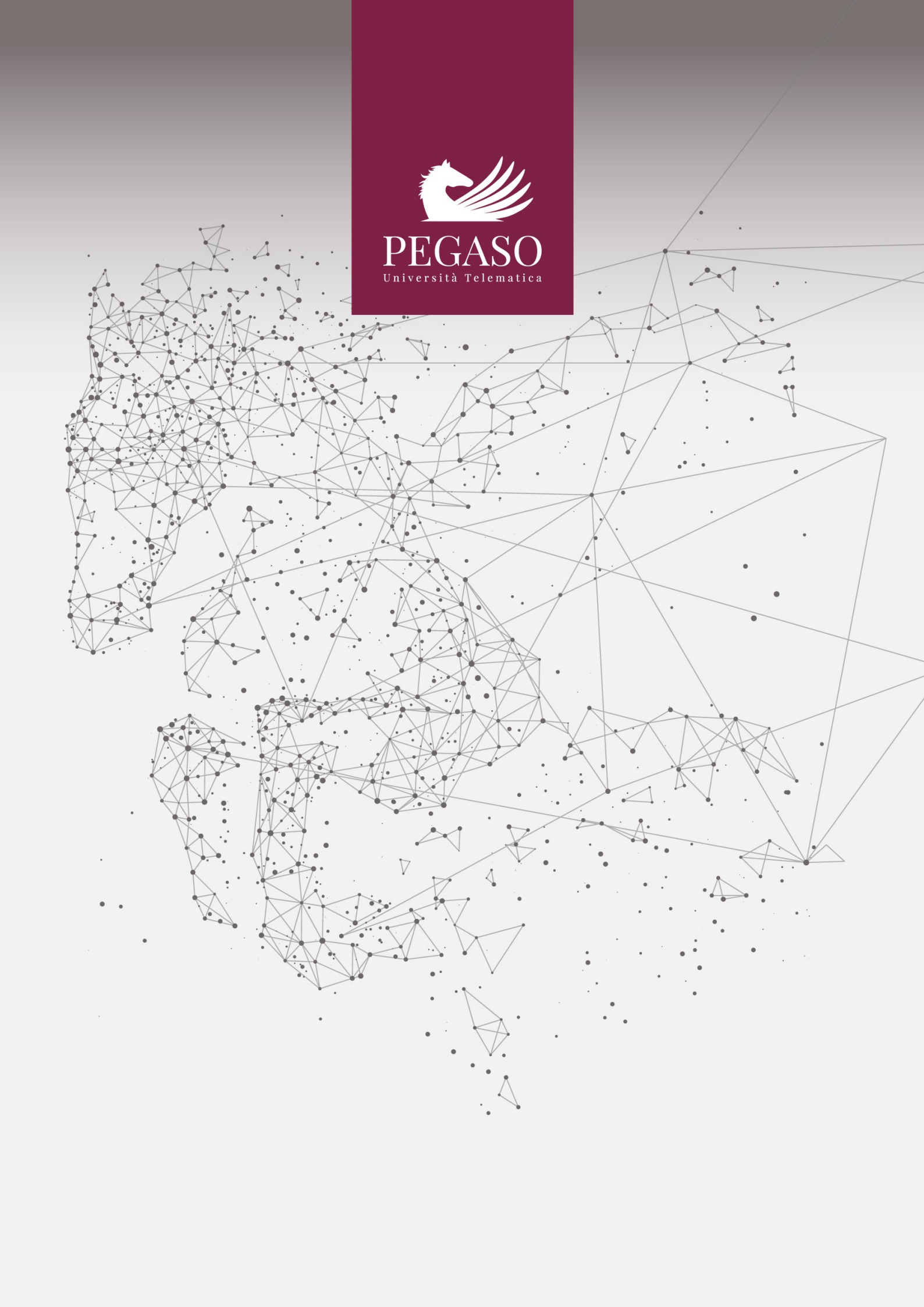

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. IMPLEMENTAZIONE DI UNA MACCHINA A STATI 3

BIBLIOGRAFIA 12

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Implementazione di una macchina a stati

Vediamo adesso un esempio che illustra i diversi modi con cui una macchina a stati può essere implementata.

Consideriamo come esempio la macchina a stati che governa il comportamento di un sistema di sicurezza (ovvero di un meccanismo di controllo) per proteggere una cassaforte nascosta all'interno di una stanza. Per arrivare alla cassaforte bisogna prima rimuovere una particolare candela dal candelabro, ma questo funziona solo se la porta è chiusa. Una volta scoperta la serratura, sarà possibile inserire la chiava per aprirla. Per aggiungere un livello extra di sicurezza, si è deciso che la cassaforte si aprirà solamente se la candela è stata rimessa al suo posto. Se il ladro dovesse ignorare questa precauzione, verrebbe liberato un coniglio mannaro che lo divorerebbe.

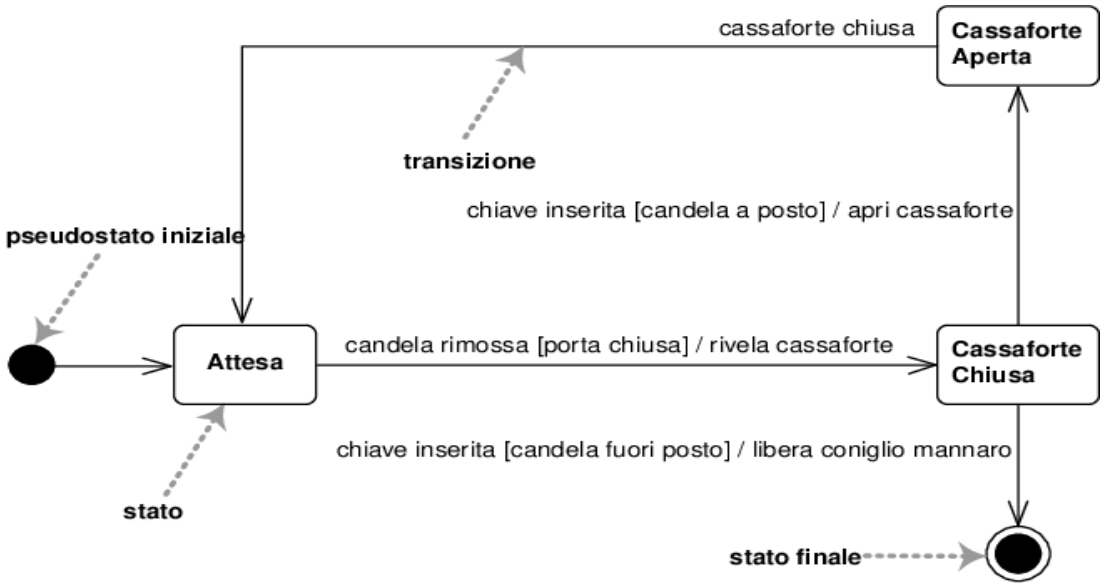


Fig. 8: Esempio di controllo del sistema di sicurezza di una cassaforte

La macchina a stati è mostrata in Fig. 8. Il sistema parte da uno stato di attesa. In questo stato con la rimozione della candela viene rimossa (evento) e se la porta è chiusa (condizione) allora il sistema rivela (azione) la cassaforte (e la usa serratura) viene rivelata e si può agire su di essa. Il sistema transisce nello stato «cassaforte chiusa». In questo stato, inserendo la chiave (evento) la cassaforte si apre (azione) purché la candela è stata rimessa al suo posto (condizione). Il sistema quindi passa allo stato «cassaforte aperta». Da questo stato in caso la cassaforte venga chiusa (evento) il sistema ritorna in stato di attesa. Se non si rispetta la procedura (ovvero si tenta di aprire la cassaforte senza non aver rimesso la candela al suo posto)

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

allora il sistema percepisce un accesso non autorizzato e quindi viene liberato un coniglio mannaro e il ciclo di controllo termina.

Vediamo adesso tre modi possibili con cui implementare questa macchina a stati: con switch nidificati, applicando una soluzione che fa uso di più classi di tipo Stato e facendo uso della ereditarietà (come descritto in un design pattern per le macchine a stati) e, infine, facendo uso di tabelle di stato.

1. Soluzione procedurale con switch nidificati

In Fig. 9 è mostrata una possibile soluzione in C# di una procedura che usa switch nidificati per gestire le transizioni di stato dell'esempio in Fig. 2:

- Lo switch principale si applica a differenti stati possibili
- Per ciascun case (stato) si applica un altro switch per ciascun evento possibile
- Soluzione più intuitiva ma la più complicata da implementare

Si tratta dell'approccio più diretto e intuitive anche se la più complicata da implementare, come dimostra la complessità del codice che gestisce questo semplice caso. E' quindi relativamente facile perdere il controllo delle modifiche apportate.

```
public void GestisciEvento (EventoCassa unEvento) {
    switch (StatoCorrente) {
        case StatoCassa.Aperta :
            switch (unEvento) {
                case EventoCassa.CassaChiusa :
                    StatoCorrente = StatoCassa.Attesa;
                    break;
            }
            break;
        case StatoCassa.Attesa :
            switch (unEvento) {
                case EventoCassa.CandelaRimossa :
                    if (isPortaAperta) {
                        RivelaCassaforte();
                        StatoCorrente = StatoCassa.Chiusa;
                    }
                    break;
            }
            break;
        case StatoCassa.Chiusa :
            switch (unEvento) {
                case EventoCassa.ChiaveInserita :
                    if (isCandeIaAPosto) {
                        ApriCassa();
                        StatoCorrente = StatoCassa.Aperta;
                    } else {
                        LiberaConiglioMannaro();
                        StatoCorrente = StatoCassa.Finale;
                    }
                    break;
            }
            break;
        }
    }
}
```

Fig. 9: Una soluzione in C# che usa switch nidificati per gestire le transizioni di stato dell'esempio in Fig. 2

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Soluzione con gerarchia di classi Stato

Si tratta di una soluzione basata sul controllo distribuito fra più classi di tipo Stato e facendo uso della ereditarietà per gestire il comportamento dei singoli stati. Questo approccio è descritto in un design pattern apposite per le macchine a stati.

Si tratta di usare una gerarchia di classi Stato che per definire le azioni da farsi per ciascuno stato in base all'evento sotto la supervisione dell'oggetto Controller..

Ad ogni stato (Attesa, Chiusa, Aperta) si fa corrispondere una classe nella gerarchia che specializza la superclasse Stato. La classe Stato è una classe astratta che implementa tutti i metodi di gestione degli eventi in modo che non facciano nulla. In ogni classe stato derivate si ridefinisce i soli metodi a cui quello stato associa una transizione

Ad ogni evento corrisponde un'operazione nell'oggetto Controller che si limita a inoltrare le chiamate alla classe di tipo Stato che in quel momento risulta come stato corrente tra gli attributi.

Anche se associa una classe ad ogni stato, la soluzione è molto elegante e limita fortemente la quantità di codice da scrivere e quindi da manutenere.

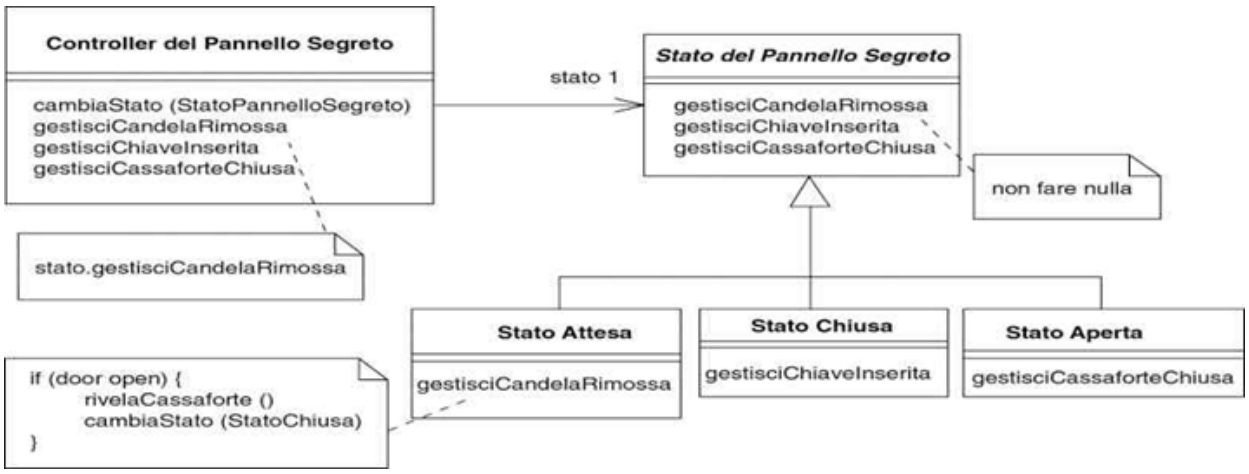


Fig. 8: Una implementazione usando una gerarchia della classe Stato

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Soluzione con tabelle di stato

La tabella di stato rappresenta l'informazione contenuta nel diagramma sottoforma di dati tabellari. Nella slide è mostrata la tabella di stato per la macchina a stati che gestisce il sistema di controllo di un pannello segreto di una cassaforte.

Una volta costruita la tabella si può costruire un interprete che la legge direttamente in fase di esecuzione, oppure usando un sistema di generazione del codice che crea le classi in base ad essa. Se la tabella è letta durante l'esecuzione non c'è bisogno di ricompilare il codice per modificala e ciò torna molto utile in molte situazioni.

Stato sorgente	Stato destinazione	Evento	Guardia	Procedura
Attesa	Chiusa	Candela rimossa	Porta aperta	Rivela cassaforte
Chiusa	Aperta	Chiave inserita	Candela a posto	Apri la cassaforte
Chiusa	Finale	Chiave inserita	Candela fuori posto	Libera il coniglio mannaro
Aperta	Attesa	Chiusura della		cassaforte

Fig. 9: Una tabella di stato per l'esempio di Fig. 2

1. Esercizi svolti

a. Invio di un SMS

Si disegni una macchina a stati per rappresentare tutti i possibili stati, e relative transizione in cui si può trovare un SMS da quando qualcuno inizia a scriverlo fino a quando il destinatario deciderà di leggerlo. Si noti che una volta inviato il messaggio potrebbe non venir spedito immediatamente, in alcuni casi sono necessari alcuni tentativi prima che la spedizione vada a buon fine.

Per prima cosa occorre individuare gli stati in cui potrebbe trovarsi il software di controllo per la scrittura e l'invio di SMS..). Lo stato rappresenta una particolare situazione o condizione in cui si può trovare il messaggio e che può influenzarne il comportamento. In questo caso gli stati si riferiscono allo stato di avanzamento del messaggio nella pipeline di preparazione e spedizione dello stesso.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Si passa da uno stato all'altro con il verificarsi di un evento che corrisponde in questo caso ad operazioni effettuate dal mittente del messaggio e/o dal software nei vari step, rispettivamente, di scrittura e spedizione del messaggio.

Come mostrato in Fig. 1, si parte da uno stato in cui tutti i campi sono da riempire denominato come "Vuoto".

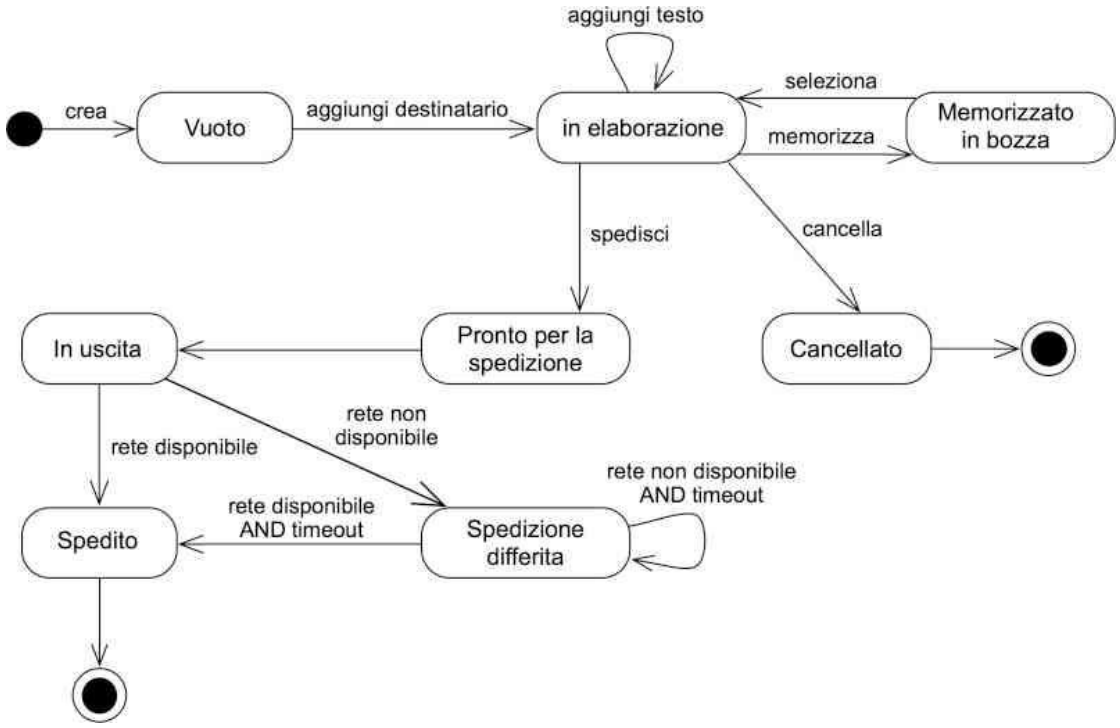


Fig. 1: Diagramma di macchina a stati relativo a "Invio di un SMS"

Lo stato successivo è "In elaborazione" che corrisponde allo stato in cui il messaggio è compilato dal mittente. In questo stato si hanno diverse azioni in funzioni dell'evento in arrive, ovvero l'opeazione compiuta dal mittente. Si passa da "Vuoto" a "In elaborazione" con il riempimento di un campo, ovvero con l'aggiunta di un destinatario.

Si resta nello stato "In elaborazione" finché il mittente digita il testo. In qualsiasi momento si passa ad uno stato "Memorizzato in bozza" se il mittente decide di memorizzare il messaggio. Da questo stato si ritorna nello stato "In elaborazione" se il mittente seleziona il messaggio per completarlo.

Dallo stato di "In elaborazione" si può anche uscire nel caso si decida di cancellare il messaggio andando nello stato "Cancellato" e quindi concludendo l'esecuzione della macchina a stati essendo completato il ciclo di vita del messaggio dal momento che è stato cancellato. Inoltre dallo stato di "In elaborazione" si può uscire quando il mittente finisce di digitare il testo del messaggio e lo spedisce. In

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

quest'ultimo caso si passa allo stato "Pronto per la spedizione" che corrisponde al trasferimento o passaggio del messaggio in un buffer affinché possa poi essere processato ai fini della spedizione effettiva. Quando ciò avviene il messaggio passa allo stato "In uscita".

La distinzione tra "Pronto per la spedizione" e "In uscita" è stata fatta per distinguere tra lo stato in cui il messaggio è in attesa di essere processato a qeullo in cui è in fase di processamento per la spedizione. Si è fatta questa distinzione perché è richiesto che si gestisca la ritrasmissione dei messaggi in fase di invio. La distinzione serve per considerare diversi casi (o meglio eventi) a partire dallo stato "In uscita". Se non era richiesto avremmo potuto usare uno stato unico del tipo "In spedizione" (non mostrato in questo diagramma) oppure omettere tali stati e usare direttamente lo stato "Spedito".

Dallo stato "In uscita" a seconda che la rete sia disponibile oppure no si passa ad uno stato "Spedito" oppure "Spedizione differita", rispettivamente. Dallo stato "Spedito" si conclude poi l'esecuzione della macchina a stati avendo raggiunto lo scopo finale. Da "Spedizione differita" si passa allo stato "Spedito" se rete si rende disponibile dopo aver atteso un intervallo di tempo scandito dallo scadere di un time-out. Si rimane nello stato "Spedizione differita" in caso contrario, ovvero allo scadere del timeout la rete risulta ancora non disponibile.

Poiché il diagramma non contiene condizioni e azioni, la tabella si presta bene alla rappresentazione di questa macchina a stati contenendo semplicemente solo le tre colonne relative allo stato iniziale, a quello finale e all'evento che causa la transizione (vedi Fig. 2). Un software può leggendo i campi della tabella interpretare il comportamento da seguire in funzione degli eventi (operazioni svolte dal mittente oppure dallo stato della rete controllato alla scadenza del timeout).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Stato inziale	Stato finale	Evento
Stato iniziale	Vuoto	crea
Vuoto	In elaborazione	aggiungi destinatario
In elaborazione	In elaborazione	aggiungi testo
In elaborazione	Memorizzato in bozza	memorizza
Memorizzato in bozza	In elaborazione	seleziona
In elaborazione	Cancellato	cancella
Cancellato	Stato finale	
In elaborazione	Pronto per la spedizione	spedisci
Pronto per la spedizione	In uscita	
In uscita	Spedito	rete disponibile
In uscita	Spedizione differita	rete non disponibile
Spedizione differita	Spedito	rete disponibile AND timeout
Spedizione differita	Spedizione differita	rete non disponibile AND timeout
Spedito	Stato finale	

Fig. 2: Macchina a stati "Invio di un SMS" in forma tabellare

b. Produzione software di una classe

Si disegni una macchina a stati per definire gli stati attraverso cui evolve una classe in fase di sviluppo, le transizioni fra questi e le diverse attività collegate. Si ipotizzi un semplice processo di sviluppo per la realizzazione di classi in un linguaggio di programmazione (ad esempio C++, C#, Java) secondo un procedimento di qualità riguardo alla produzione di software. Ogni classe viene pensata dall'analisi dei requisiti, progettata, sviluppata, collaudata come elemento isolato tramite test di unità, integrata nel sistema software complessivo, rilasciata e poi manutenuta opportunamente. Se il test di unità rileva un numero di anomalie superiore a N, la classe deve subire un processo di manutenzione straordinaria prima di essere sottoposta a un nuovo test di unità e pensare alla possibile integrazione con il resto del sistema. Ogni attività di test di unità deve ripartire da zero (non può mai usare test o risultati acquisiti in precedenza). Se la stessa classe richiede più di tre manutenzioni straordinarie, l'implementazione è archiviata e si dà inizio a un nuovo sviluppo. Per alleviare questo problema, durante la fase di sviluppo, esiste un proceso di raccolta dati e analisi della qualità continuo: i dati raccolti potrebbero essere utili durante le diverse fasi di collaudo e manutenzione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

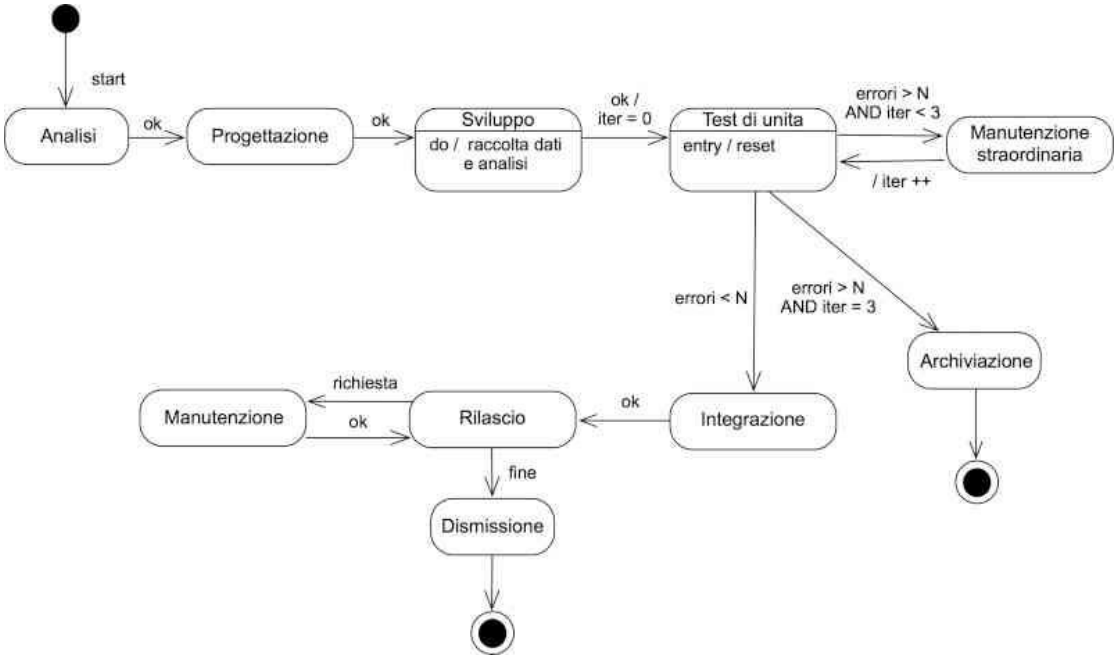


Fig. 3: Diagramma di macchina a stati relativo a "Produzione software di una classe"

Come mostrato in Fig. 3, gli stati corrispondono a ciascun stadio in cui si può trovare una classe nel processo di produzione software: Analisi, Progettazione, Sviluppo, Test di unità, Manutenzione straordinario, Archiviazione, Integrazione, Rilascio, Manutenzione, Dismissione.

Finita ciascuna fase a partire dallo stato Analisi si evolve nello stato dello stadio successivo fino a Test di unità. Da Test di unità si passa allo stato successivo in base all'esito dei test ovvero il numero di errori. Se il test di unità rileva un numero di anomalie superiore a N, la classe deve subire un processo di manutenzione straordinaria prima di essere sottoposta a un nuovo test di unità fino ad un massimo di 3 volte. Superato questo limite di iterazioni la classe viene archiviata definitivamente e il diagramma si conclude. Se il test di unità rileva un numero di anomalie minore a N la classe viene integrata nel sistema software complessivo (Integrazione), rilasciata (Rilascio) e poi manutenuta opportunamente (Manutenzione). L'evento ok è una semplificazione per rappresentare la terminazione della fase di lavorazione e quindi l'abilitazione al passaggio di stato. In modo simile, l'evento richiesta rappresenta la richiesta di manutenzione e l'evento fine la decisione di sostituire completamente l'elemento rilasciato con il conseguente passaggio allo stato Dismissione.

a. Televisore moderno

Si disegni una macchina a stati per rappresentare tutti i possibili stati, e relative transizione in cui si può trovare un televisore moderno considerato che: può utilizzare diverse sorgenti: il segnale tradizionale,

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

il digitale terrestre, la consolle per i video giochi e il lettore DVD. L'utente può cambiare la sorgente utilizzando il tasto selSrg del telecomando: ogni pressione del tasto provoca un passaggio all'elemento successivo della lista sopra indicata e dall'ultimo si torna al primo. Chiaramente, in prima approssimazione si può dire che la televisione è spenta oppure accesa: se accesa vorremmo essere in grado di tenere traccia della sorgente in uso; quando si spegne il televisore l'apparecchio memorizza la configurazione corrente per poterla riproporre all'utente non appena dovesse riaccenderlo.

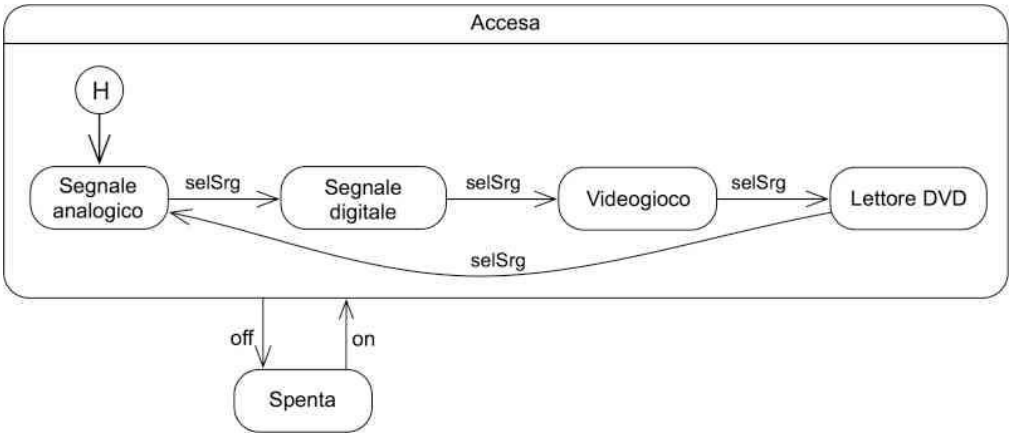


Fig. 4: Diagramma di macchina a stati relativo a "Televisore moderno"

Come mostrato in Fig. 4, in questo caso si fa uso dei superstati per raccogliere dei sotto-stati sotto un comportamento comune che in questo caso è rappresentato dallo stato Accesa. Infatti la televisione può essere spenta o accesa e quando è accesa il suo stato può passare dal funzionare con segnale analogico, segnale digitale, come videogioco o come lettore DVD. Il simbolo con "H" rappresenta uno "pseudostato di storia" che in questo diagramma indica che la scelta sul segnale viene ricordata e quindi, quando si accende la televisione, la scelta si pone nello stato in cui era quando la tv è stata spenta. La freccia uscente dal pseudostato indica in quale stato bisogna porsi alla prima accensione, quando non c'è una storia passata del sistema.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- A. Cockburn, Writing Effective Use Cases, 2001, Reading, M.A.: Addison-Wesley
- M. Fowler, UML Distilled, Pearson, Quarta edizione
- R. S. Pressman, Principi di Ingegneria del software, McGraw-Hill, Quinta edizione
- C. Larman, Applicare UML e i pattern, Pearson, Quinta edizione

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).